

Engenharia de Software

Prof. Diego Max

CURSO: DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

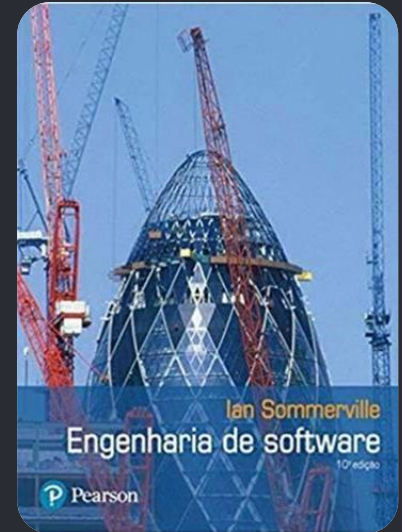
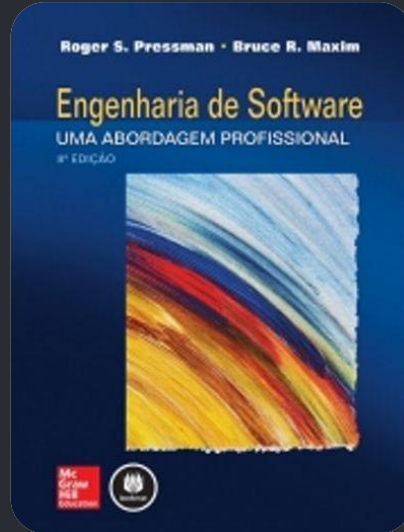
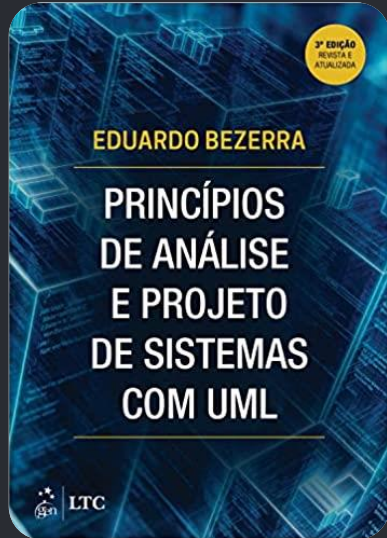
Objetivos da Disciplina

- ✓ Identificar as características de Sistemas de Informação, seus tipos, viabilidade técnica, características de custo, valor e qualidade da informação.
- ✓ Explicar as características de um sistema, seus componentes e relacionamentos.
- ✓ Compreender o ciclo de vida utilizando concepções do modelo cascata.
- ✓ Utilizar conceitos da UML na análise de requisitos e na elaboração de diagramas focando na modelagem de sistemas.

Ementa da Disciplina

- ✓ Introdução à Análise de Sistemas.
- ✓ Modelos de Ciclo de Vida de Software.
- ✓ Modelos de Processos de Desenvolvimento de Software (Modelo em Cascata, Espiral e Prototipagem).
- ✓ Definição e classificação de Requisitos de Software (funcionais e não funcionais).
- ✓ Técnicas de Levantamento de Requisitos.
- ✓ Modelo de Negócios aplicado ao levantamento de Requisitos (Canvas).
- ✓ Estudo de Viabilidade.
- ✓ Técnicas de documentação.
- ✓ Metodologias para desenvolvimento de sistemas.

Bibliografia da Disciplina



Aula 01:

Introdução à Engenharia de Software

Fundamentos, Processos e Aplicações

Objetivos da Aula

Ao final desta aula, você será capaz de responder: *o que é Engenharia de Software, por que ela surgiu e como ela se aplica no desenvolvimento para múltiplas plataformas.*

1

Conceito e Origem

Compreender o que é Engenharia de Software e como a disciplina foi criada.

2

Crise do Software

Entender o contexto histórico que motivou o surgimento da área nos anos 1960.

3

Ciclo de vida e Modelos de Processo

Conhecer o ciclo de vida de um software e os principais modelos de processo: cascata, incremental e metodologias ágeis.

4

Aplicação Multiplataforma

Relacionar os princípios de engenharia ao desenvolvimento para web, mobile e desktop.

O que é Engenharia de Software?

"A aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software." — **IEEE, 1990**

Segundo Sommerville (2019), a Engenharia de Software é uma disciplina de engenharia relacionada a **todos os aspectos da produção de software** — do levantamento de requisitos até a manutenção pós-entrega. Ela vai muito além de "escrever código": envolve planejamento, análise, projeto, testes e manutenção contínua, com foco em qualidade e processo estruturado.

O IEEE (Institute of Electrical and Electronics Engineers, ou Instituto de Engenheiros Eletricistas e Eletrônicos)

Software vs. Programa

Existe uma distinção fundamental que todo desenvolvedor precisa conhecer: um **programa** é apenas parte de um produto de software.

Programa / Código

Conjunto de instruções executáveis escritas em uma linguagem de programação. É o código-fonte em si — necessário, mas insuficiente.

Software

Programa + documentação técnica + dados de configuração + processos associados. Um produto completo de engenharia.

Desenvolver software é um trabalho de engenharia porque envolve muito mais do que linhas de código: inclui manuais, testes, configurações e toda a infraestrutura que garante o funcionamento do produto.

Por que Software é Difícil de Construir?

Frederick Brooks, em seu artigo clássico "**No Silver Bullet**" (1987), argumenta que *não existe solução mágica* para os problemas do desenvolvimento de software — a complexidade é essencial à sua natureza.



Complexidade Inerente

A complexidade não desaparece, apenas se transforma.
Diferente de outras engenharias, o software não tem equivalente físico simples.



Requisitos em Constante Mudança

O mundo dos negócios muda — e o software precisa acompanhar. Novos requisitos surgem durante o próprio desenvolvimento.



Intangibilidade

Software não se vê nem se toca. Isso dificulta a percepção de progresso e a identificação de problemas durante o desenvolvimento.



Fatores Humanos

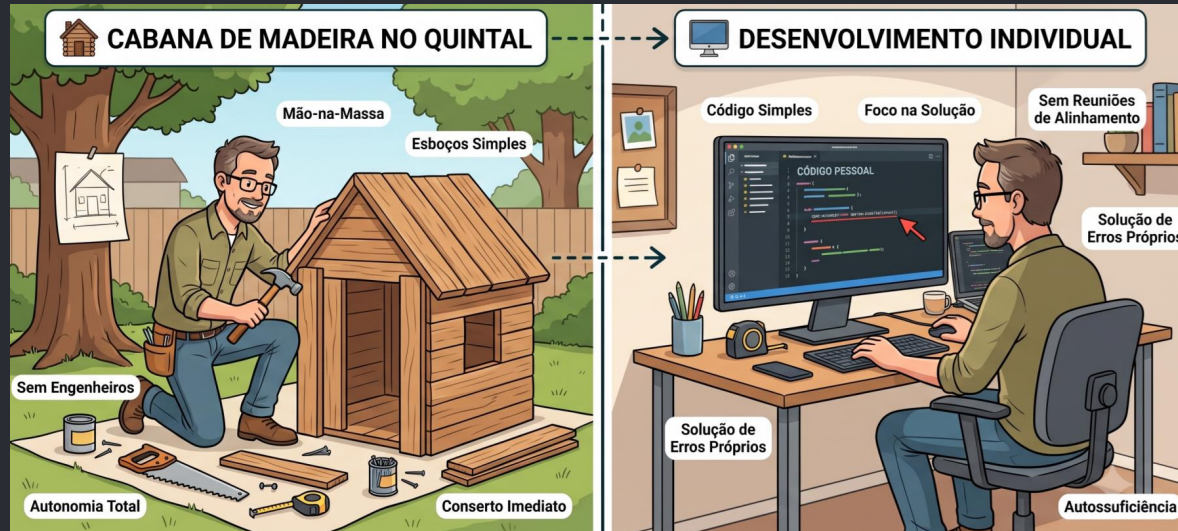
Comunicação falha, equipes distribuídas e divergências de interpretação são causas frequentes de falha em projetos.



A Casa de Madeira e o Arranha-Céu

A Engenharia de Software não é apenas sobre escrever código. Ela existe para transformar a programação, que pode ser um ato isolado, em um processo profissional, escalável e colaborativo.

Programação Individual (A Casa de Madeira): É como construir uma pequena cabana no quintal. Você não precisa de plantas arquitetônicas complexas, engenheiros ou reuniões de alinhamento. Você simplesmente pega os materiais e começa a construir. Se houver um erro, você mesmo entende e conserta.

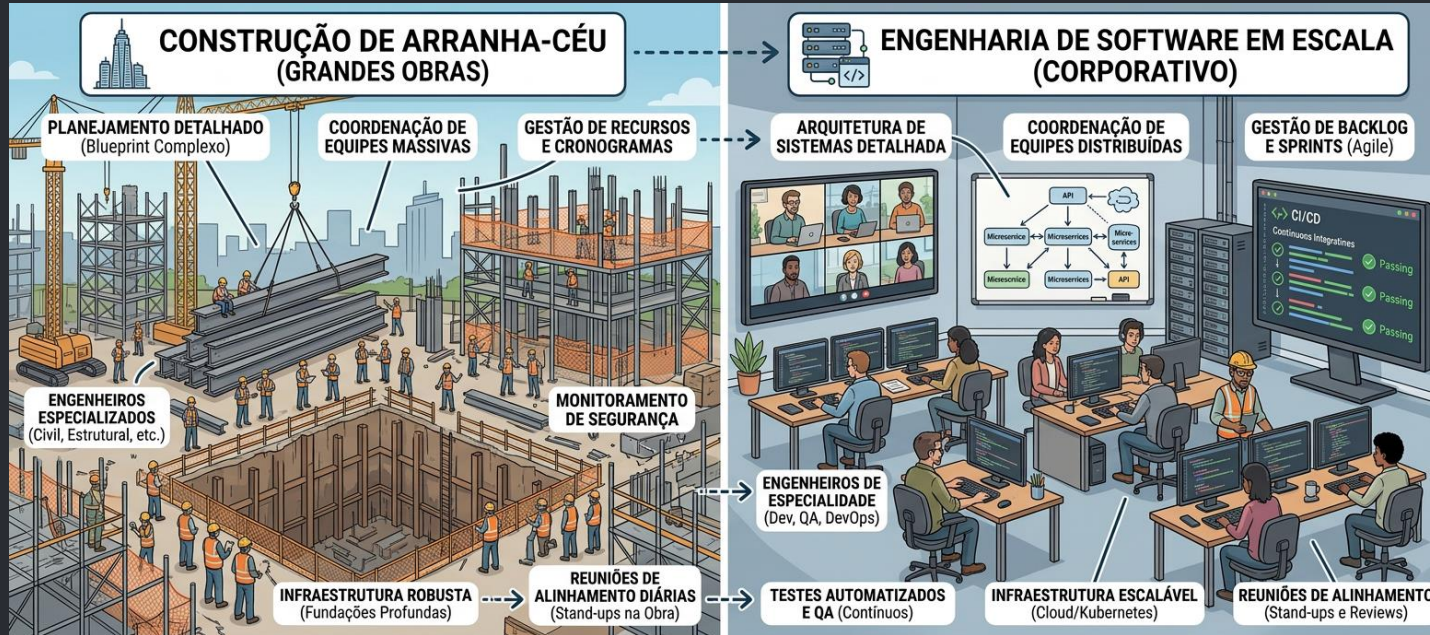


A Casa de Madeira e o Arranha-Céu

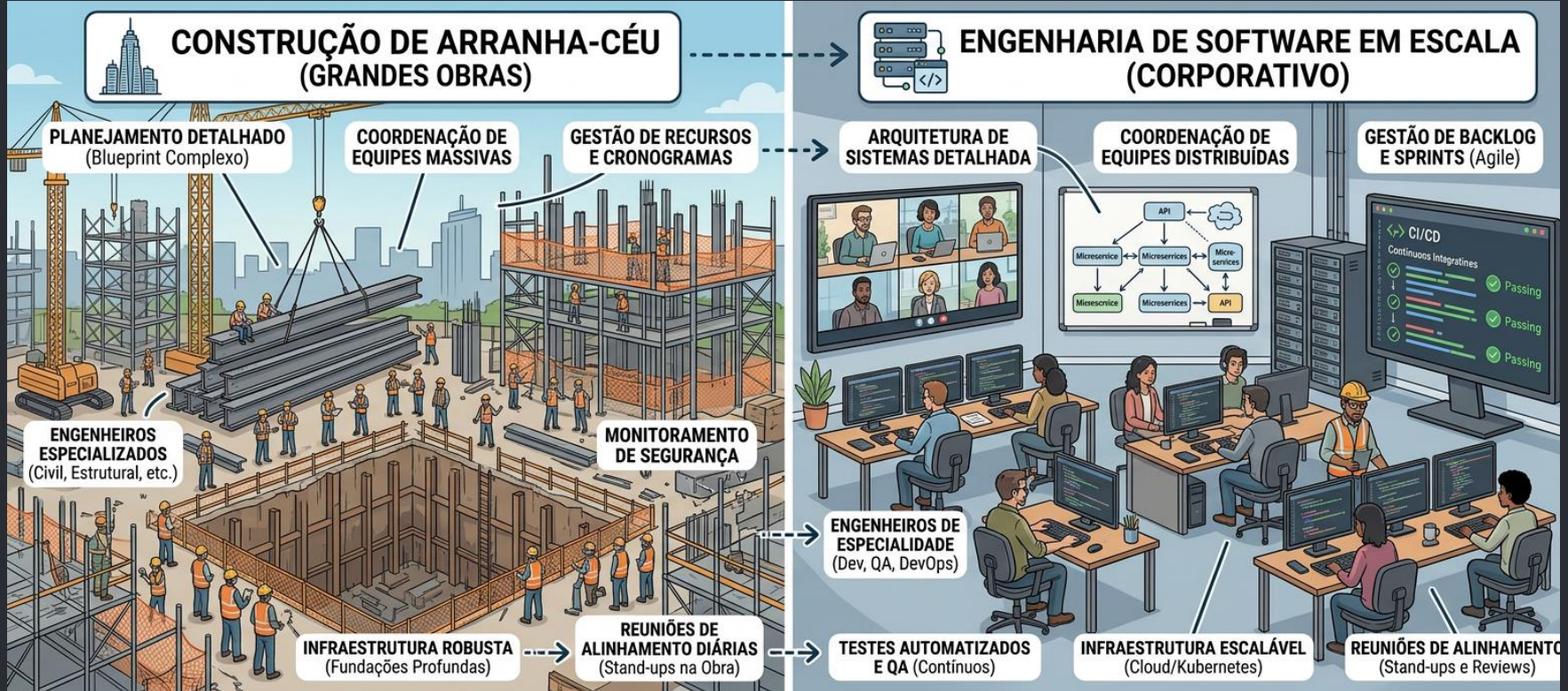


A Casa de Madeira e o Arranha-Céu

Engenharia de Software (O Arranha-Céu): É como construir um prédio comercial. Você não pode simplesmente começar a empilhar tijolos. É necessário estudar o solo, desenhar plantas rigorosas, coordenar dezenas de profissionais e garantir que a estrutura seja segura para milhares de pessoas.



A Casa de Madeira e o Arranha-Céu





Desafios da Engenharia de Software

Apesar disso, a engenharia de software ainda é criticada como inadequada para o desenvolvimento de software moderno. Muitas dessas falhas são consequência de fatores como:

Complexidade crescente dos sistemas: À medida que as novas técnicas de engenharia de software nos ajudam a construir sistemas maiores e mais complexos, as demandas mudam.

Os sistemas precisam ser construídos e distribuídos com mais rapidez; é necessário que sejam ainda maiores e mais complexos; e eles precisam ter novas capacidades, que antes eram consideradas impossíveis.

Desafios da Engenharia de Software

Exemplo:

Anteriormente: O consumo de mídia dependia de mídias físicas (DVDs, CDs) gerenciadas por sistemas de estoque simples, ou de downloads muito lentos na internet.

Atualmente: Com o avanço das redes e dos softwares, o usuário não queria mais esperar. A demanda passou a ser: *"Quero abrir um aplicativo no celular e assistir a um filme em resolução 4K de forma instantânea, sem travar, enquanto o sistema me recomenda exatamente o que eu quero ver usando Inteligência Artificial"*.

Para entregar essa "mágica", empresas como a Netflix construíram sistemas de uma complexidade colossal, distribuídos em servidores no mundo todo.



MARCO HISTÓRICO

A Crise do Software

Em **1968**, a OTAN promoveu uma conferência histórica para discutir um problema alarmante: os projetos de software da época estouravam prazos, custavam muito mais do que o previsto e frequentemente não funcionavam como esperado. Esse fenômeno ficou conhecido como a "**Crise do Software**".

Atrasos

Projetos entregues muito além do prazo previsto.

Estouro de Orçamento

Custos reais muito acima das estimativas iniciais.

Baixa Qualidade

Sistemas cheios de erros e com difícil manutenção.

Origem do Termo

A resposta foi propor o rigor da engenharia para o desenvolvimento de software.

“Crise do Software” e o Desafio da Escalabilidade

O conceito de **Engenharia de Software** foi proposto nessa conferência, realizada para discutir o que então se convencionou chamar de "crise do software" (NAUR; RANDELL, 1969).

Até àquela época, o desenvolvimento de sistemas dependia fortemente de abordagens artesanais e individuais.

Contudo, tornou-se evidente que estas metodologias isoladas não escalavam para sistemas de software grandes e complexos.

“Crise do Software” e o Desafio da Escalabilidade

Exemplo:

Imagine um software acadêmico construído no final dos anos 1960 por um único programador que utilizava regras próprias e sem qualquer padronização. Enquanto o sistema geria o registro de apenas 50 estudantes, funcionava perfeitamente.

No entanto, quando a instituição cresceu para 5.000 estudantes e exigiu a integração de um módulo financeiro, a arquitetura original colapsou.

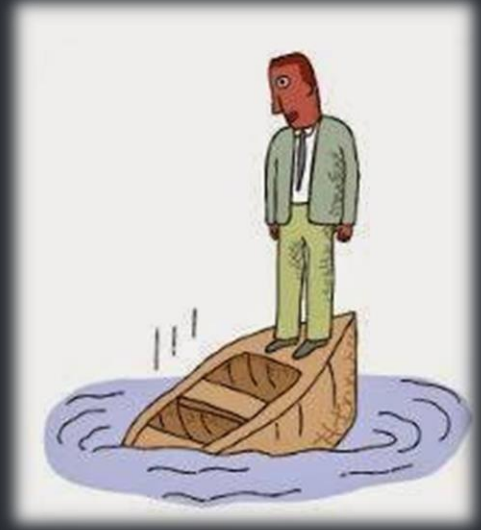
O sistema tornou-se excessivamente lento, sofria falhas constantes e a introdução de novas funcionalidades demorava meses, demonstrando que o modelo artesanal não era escalável.

“Crise do Software” e o Desafio da Escalabilidade

Consequência:

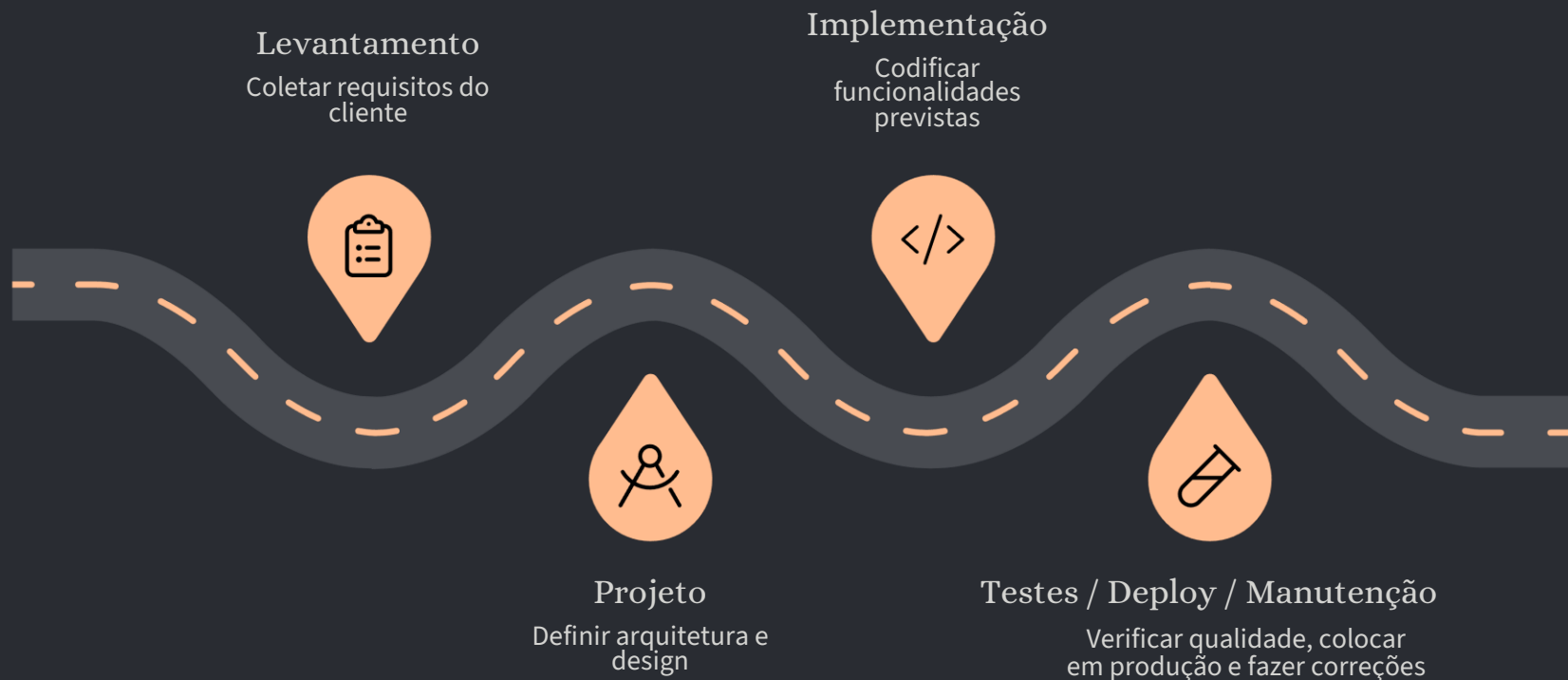
Como consequência direta desta falta de metodologia, os sistemas da época sofriam de três problemas crônicos:

- não eram confiáveis;
- custavam muito mais do que o previsto;
- e eram entregues com atrasos significativos.



Ciclo de Vida do Software

Todo software, independentemente do modelo de processo adotado, percorre um conjunto de fases fundamentais. Dominar todas elas é o que diferencia um **engenheiro de software** de um **simples programador**.



A maior parte dos cursos de programação foca na fase de **implementação**, mas ela representa apenas uma das seis etapas. Profissionais completos precisam compreender e atuar em todo o ciclo.

Levantamento de requisitos

Nesta fase inicial, a equipe busca entender as necessidades do cliente ou usuário final, definindo o que o sistema deve fazer.

Envolve reuniões, entrevistas e documentação para mapear funcionalidades, restrições e expectativas antes de qualquer linha de código ser escrita.

Exemplo:

Ao desenvolver um aplicativo de delivery, a equipe entrevista donos de restaurantes e entregadores para descobrir que o app precisa calcular frete automaticamente e notificar o cliente em tempo real.



Análise e projeto (design)

Com os requisitos definidos, é hora de planejar como o software será construído. Aqui se define a arquitetura do sistema, o modelo de dados, as interfaces e os componentes técnicos, criando um *blueprint* que guiará a implementação.

Exemplo:

Para o mesmo app de delivery, os arquitetos decidem usar uma arquitetura de microsserviços, separando o módulo de pedidos, o de pagamentos e o de rastreamento, e desenham as telas no Figma antes de codificar.

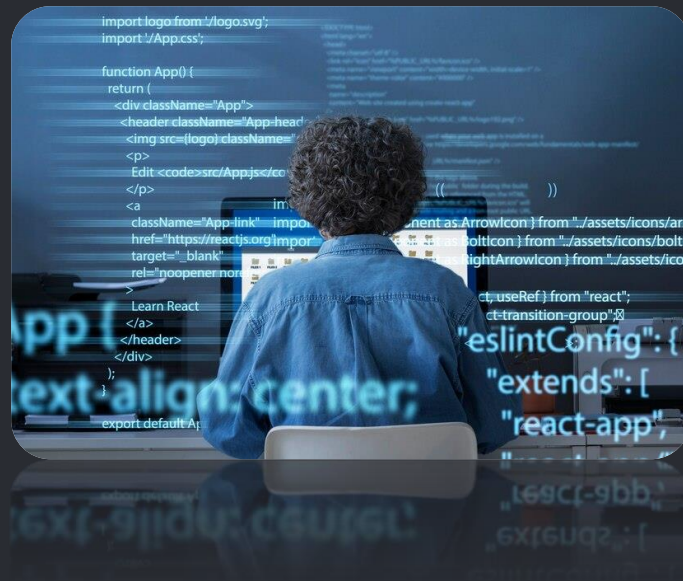


Implementação (codificação)

É a etapa em que os desenvolvedores efetivamente escrevem o código, transformando o design em um sistema funcional. Segue-se boas práticas de programação, padrões de código e, geralmente, o uso de **controle de versão**.

Exemplo:

Os programadores criam a funcionalidade de rastreamento em tempo real usando um serviço de geolocalização, versionando o código no Git e integrando com a API do módulo de pedidos.

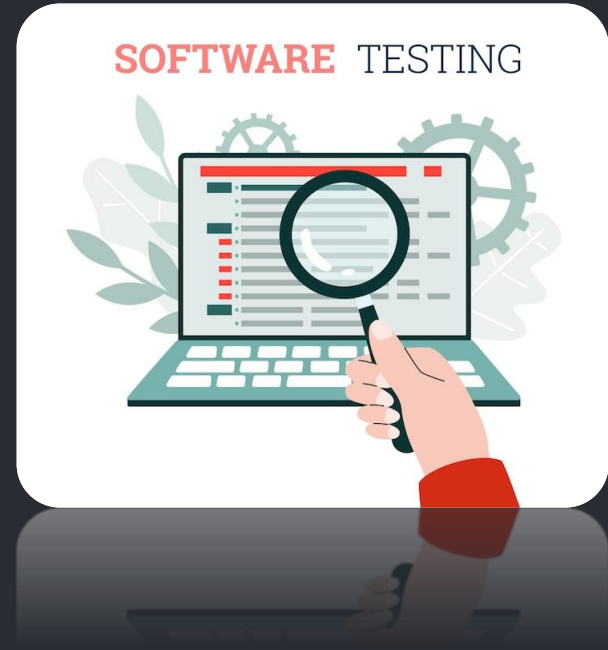


Testes

O software é validado para garantir que funciona corretamente e atende aos requisitos estabelecidos. Envolve testes unitários, de integração, de sistema e, muitas vezes, testes de aceitação com o usuário, identificando bugs antes do lançamento.

Exemplo:

A equipe de QA (Qualidade de Software) testa se o cálculo de frete funciona corretamente para diferentes distâncias e simula quedas de conexão para verificar se o rastreamento se recupera sem travar o app.



Implantação (deploy)

Depois de testado, o software é colocado em produção, ficando disponível para os usuários finais. Pode envolver configuração de servidores, migração de dados e monitoramento inicial para garantir que tudo funcione como esperado no ambiente real.

Exemplo:

O app de delivery é publicado na Google Play Store e na App Store, e os servidores são configurados na nuvem para suportar o volume esperado de pedidos no lançamento.



Manutenção

Após o lançamento, o software precisa de ajustes contínuos: correção de bugs, atualizações de segurança, melhorias de desempenho e adição de novas funcionalidades conforme as necessidades evoluem. É a fase mais longa do ciclo de vida.

Exemplo:

Alguns meses depois, a equipe corrige um bug que fazia o app travar em celulares Android mais antigos e adiciona uma nova funcionalidade de avaliação de entregadores, solicitada pelos usuários.



Engenharia de Requisitos

Antes de escrever qualquer linha de código, é fundamental entender **o que o sistema precisa fazer**. Boa parte dos fracassos em projetos de software origina-se de requisitos mal levantados ou mal documentados — não de erros de programação.

Tipos de Requisitos

Funcionais

O que o sistema **deve fazer**: funcionalidades, regras de negócio, comportamentos esperados.

Não Funcionais

Como o sistema **deve se comportar**: desempenho, segurança, escalabilidade, portabilidade.

Técnicas de Levantamento

- **Entrevistas** com stakeholders e usuários finais
- **Questionários** para coletar informações de forma escalável
- **Prototipagem** para validar ideias antes do desenvolvimento
- **Rastreabilidade** para manter o vínculo entre requisito e implementação

Requisitos funcionais (o que o sistema deve fazer)

- O sistema deve permitir que o usuário se cadastre com e-mail e senha
- O sistema deve permitir adicionar produtos ao carrinho de compras
- O sistema deve calcular automaticamente o frete com base no CEP informado
- O sistema deve enviar um e-mail de confirmação após a finalização do pedido
- O sistema deve permitir que o administrador cadastre, edite e exclua produtos
- O sistema deve gerar um relatório de vendas mensal em PDF
- O usuário deve poder recuperar sua senha via e-mail

Requisitos não funcionais (como o sistema deve ser ou se comportar)

- O sistema deve responder a qualquer requisição em até 2 segundos
- O sistema deve suportar 10 mil usuários simultâneos sem degradação de performance
- Os dados dos usuários devem ser criptografados com AES-256
- O sistema deve ter disponibilidade de 99,9% (uptime)
- A interface deve ser responsiva e funcionar corretamente em dispositivos móveis
- O sistema deve estar em conformidade com a LGPD
- O código deve seguir os padrões de arquitetura definidos para facilitar manutenção futura
- Backups do banco de dados devem ser realizados a cada 24 horas

Engenharia de Software

Prof. Diego Max

CURSO: DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA